Neural Network-Assisted Receiver Design via Learning Trellis Diagram Online

Jie Yang, *Student Member, IEEE*, Qinghe Du, *Student Member, IEEE*, and Yi Jiang, *Member, IEEE*

***Abstract*— This paper studies machine learning-assisted optimal receivers for a communication system with memory, which can be modelled by a trellis diagram. The prerequisite of the optimal receiver is to obtain the likelihoods of the received samples under different state transitions. We propose to learn the trellis diagram real-time using an artificial neural network (ANN) trained by a pilot sequence. This approach, termed as the online learning trellis diagram (OLTD), requires neither the channel state information (CSI) nor statistics of the noise, and can be incorporated into the classic Viterbi and the BCJR algorithm. In a channel with non-Gaussian noise, the OLTD method can significantly outperform the model-based methods that use Gaussian assumption. It requires much less training overhead than the state-of-the-art ANN-assisted methods. As an illustrative example, the OLTD-based BCJR is applied to a Bluetooth low energy (BLE) receiver trained only by a 256-sample pilot sequence. Moreover, the OLTD-based BCJR can accommodate for turbo equalization. As an interesting by-product, we propose an enhancement to the BLE standard by introducing a bit interleaver to its physical layer; the resultant improvement of the receiver sensitivity can make it a better fit for some Internet of Things (IoT) communications.**

***Index Terms*— Neural network, BCJR algorithm, Viterbi algorithm, turbo equalization, Bluetooth.**

I. Introduction

RELIABLE detection and decoding is essential for any communication systems. For a single-carrier system in an inter-symbol interference (ISI) channel, which can be modeled by a finite-state trellis diagram, the classic design of a maximum likelihood (ML) receiver relies on the likelihood function of the state transitions in the trellis diagram [1]. In this paper, we propose to learn the likelihood function using an artificial neural network (ANN) based on a pilot sequence of moderate length, requiring neither the channel state information (CSI) nor the statistics of the noise.

As related works, machine learning-assisted wireless communications have attracted broad attentions in recent years [2], [3], [4], [5], [6], such as machine learning-assisted channel decoding [7], [8], [9], [10], [11] and symbol detection in a multi-input multi-output (MIMO) system [12], [13], [14], [15]. Deep learning algorithms are found to be more effective in addressing the difficult problem of symbol detection with incomplete CSI [16], [17], [18], [19]. The aforementioned works, however, attempt to substitute a whole communication system by an ANN, which requires a large amount of training data (a lengthy pilot sequences of tens of thousand samples or more), way too much to be practically feasible. The notion of the model-driven method, which combines machine learning techniques and model-based expert knowledge, is introduced in [20] to better incorporate machine learning techniques into a communication system. The efficacy of this type of model-driven methods is demonstrated in [21] and [22].

The recent works by Shlezinger et. al. [23], [24] advocate to use a (relatively simple) neural network to substitute only the channel-dependent part of the Viterbi [25] and the BCJR receiver [26]. The resultant algorithms, the so-termed ViterbiNet and BCJRNet, train a neural network to learn the *a posteriori* probability (APP) of the state transitions given the received samples and use the finite mixture model (FMM) [27] to estimate the marginal probability density of the channel output, assuming that the channel noise is Gaussian. Therefore, the ViterbiNet and BCJRNet require only a few thousand training samples, which are significantly less than that in [16] and [17], but may still be too much to be practically competitive compared with the conventional model-based method.

In this paper, we first consider the same problem as addressed in [23] and [24]. We adopt the notion of integrating a simple neural network into a communication system as advocated by Shlezinger et. al. and propose a new method, termed as the online learning trellis diagram (OLTD). The OLTD can also be integrated into the Viterbi algorithm [25] and the BCJR algorithm [26]. The resultant OLTD-based Viterbi and OLTD-based BCJR differ from the ViterbiNet and BCJRNet in that the ANN is used to learn the likelihoods of the received samples under different state transitions, rather than the APPs of the state transitions given the received samples. Owing to this critical difference, the OLTD enjoys some important advantages. First, the pilot sequence required by the OLTD method is one order of magnitude less than that needed in the state-of-the-art ANN-assisted methods [23], [24]. Second, the OLTD does not need to assume the statistics of the channel noise nor to estimate the marginal distribution of the channel outputs as the ViterbiNet and BCJRNet do. Third, it can accommodate for iterative turbo receiver

Manuscript received 13 December 2021; revised 17 April 2022, 27 July 2022, and 5 September 2022; accepted 28 October 2022. Date of publication 3 November 2022; date of current version 19 December 2022. The work was supported by National Natural Science Foundation of China Grant No. 61771005. An earlier version of this paper was presented in part at the IEEE/CIC International Conference on Communications (ICCC) 2021, Xiamen, China [DOI: 10.1109/ICCC52777.2021.9580317]. The associate editor coordinating the review of this article and approving it for publication was A. Alkhateeb. *(Corresponding author: Yi Jiang.)*

Jie Yang is with the Department of Design, Huawei Ltd., Shanghai 201206, China (e-mail: jieyang19@fudan.edu.cn).

Qinghe Du and Yi Jiang are with the Key Laboratory for Information Science of Electromagnetic Waves (MoE), School of Information Science and Technology, Fudan University, Shanghai 200438, China (e-mail: qhdu20@fudan.edu.cn; yijiang@fudan.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCOMM.2022.3219141.

Digital Object Identifier 10.1109/TCOMM.2022.3219141

0090-6778 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

for much improved performance, whereas the state-of-the-art ANN-assisted methods cannot.

To show the practical feasibility of the proposed OLTD method, we apply it to the physical layer (PHY) of the Bluetooth Low Energy (BLE) protocol [28]. A BLE system adopts the coded Gaussian frequency shift keying (GFSK) modulation, which belongs to the family of continuous phase modulation (CPM) [29]. By modeling the GFSK modulation process with a trellis diagram, we employ the OLTD method to learn the likelihoods of the state transition associated with each received sample based on the 256-sample pilot sequence as regulated in the BLE protocol [30, Ch. 6, PartB], and then use the conventional Viterbi or BCJR algorithm to recover the information bits. This study shows that our proposed neural network assisted receiver can work for a real wireless protocol.

We further introduce a bit-interleaver to the coded GFSK system, for which we combine seamlessly the OLTD method with turbo equalization [31], [32], [33] to achieve significantly improved receiver sensitivity than the conventional Bluetooth. Unlike the previous neural network based methods that unfold each iteration with one layer of the neural network [9], [11], the OLTD-based method obtains the likelihood of each state-transition in the trellis diagram only once through the whole iterative process. In contrast, the BCJRNet [24] assumes that all the coded bits have equal probability and therefore is not applicable to turbo equalization as explained in Section IV-B.

The contributions of this paper are summarized as follows:

- We show that what a neural network needs to learn about the trellis diagram is *the normalized likelihoods* of the received sample, rather than the *a posteriori* probabilities of state transitions as adopted in the BCJRNet/ViterbiNet [23], [24]. Owing to this insight, our proposed OLTD method is computationally simpler, robuster, and more versatile than the state-of-the-art ANN-assisted methods. Simulations show that adopting a small scale ANN with only one hidden-layer is sufficient for the OLTD.
- In contrast to the BCJRNet and ViterbiNet [23], [24], our proposed OLTD does not need to compute the marginal distribution of received samples, nor does it assume the statistics of the channel noise or the *a priori* probabilities of the transmitted symbols; thus, the OLTD is computationally much simpler and robuster.
- The OLTD-based BCJR method can be seamlessly incorporated into a turbo equalizer for significantly improved performance. But the BCJRNet method cannot, as simulated and analyzed in Section IV-B. In this sense, our proposed method is more versatile.
- Based on a pilot sequence of a practical length, our neural network-based method can outperform the model-based approaches in channels with non-Gaussian noise or interference, as illustrated by the numerical simulations.
- As an interesting by-product, this study indicates a possible enhancement to the BLE standard, i.e., to introduce a bit interleaver between the convolutional encoder and the GFSK modulator for much improved reliability, which can make BLE a better candidate for some Internet of Things (IoT) communications [34].

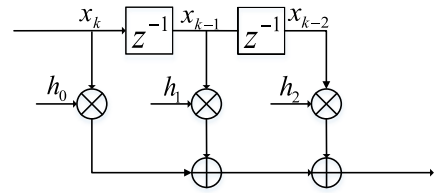

Fig. 1. The tapped delay line model for an ISI channel with length $L = 3$.

The remainder of this paper is organized as follows: Section II introduces the system model and briefly review the BCJR algorithm and Viterbi algorithm. Section III explains how to train the OLTD and how the OLTD-based BCJR/Viterbi algorithm works online. Section IV introduces the OLTD-based turbo equalization and its application to a bit-interleaved coded GFSK system. In Section V, the simulation results are given to verify the effectiveness of OLTD-based method and show the superior performance of the proposed OLTD-based BCJR/Viterbi and OLTD-based turbo algorithm. The conclusion is given in Section VI.

II. System Model and Preliminaries

A. An ISI Channel Model

The received signal in an ISI channel can be expressed as

$$y_k = \sum_{l=0}^{L-1} h_l x_{k-l} + z_k, \quad k = 1, 2, \ldots, N, \tag{1}$$

where L is the channel length, $h_l, 0 = 1, \cdots, L-1$ are the channel coefficients, and z_k denotes the i.i.d additive noise, which is not necessarily Gaussian.

The ISI channel can be modeled as a tapped delay line as shown in Fig. 1. Owing to the shift register structure, the channel can be modelled by a trellis diagram [1].

As the transmitted symbols are drawn from a set of $\mathcal{X}$, the ISI channel modeled in (1) can be represented by a trellis diagram consisting of a set $\mathcal{S}$ of the states with cardinality $|\mathcal{S}| = |\mathcal{X}|^{L-1}$. Denote the state set at time k as s_k, which corresponds to the combination of $L-1$ symbols $x_{k-L+1}, \ldots, x_{k-1}$. The state transitions $s_k \rightarrow s_{k+1}$ is associated with the output signal $v_k = \sum_{l=0}^{L-1} h_l x_{k-l}$. As an illustrative example, consider an ISI channel with coefficients $h_0 = 0.407$, $h_1 = 0.815$, and $h_2 = 0.407$ and the binary phase shift key (BPSK) signal as the input. The trellis consists of $|\mathcal{S}| = 4$ states, with $r_0 = (-1, -1)$, $r_1 = (+1, -1)$, $r_2 = (-1, +1)$, $r_3 = (+1, +1)$, as shown in Fig. 2. The numbers on the branches represent u_k and v_k. Take the state r_0 for example: if the input symbol $u_k = -1$, the branch output $v_k = -h_0 - h_1 - h_2 = -1.629$; if $u_k = +1$, $v_k = h_0 - h_1 - h_2 = -0.815$.

B. The GFSK Modulation

The trellis diagram can also be used to model a modulation with memory. As an example, we review the GFSK, which is used in the PHY of Bluetooth [28]. The GFSK with bandwidth $1/T$ sampled at $t = nT$ is

$$x_n = e^{j\phi_n}, \quad n = 1, 2 \cdots, N, \tag{2}$$

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

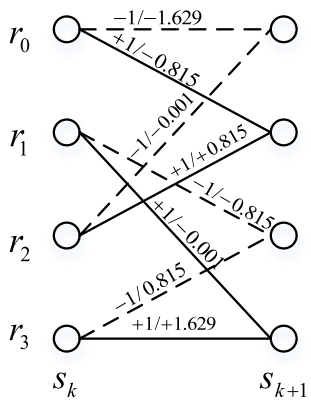

Fig. 2. The trellis diagram of the ISI channel with length $L = 3$ and the BPSK input; the solid line represents $u_k = 1$, the dashed line represents $u_k = 0$.

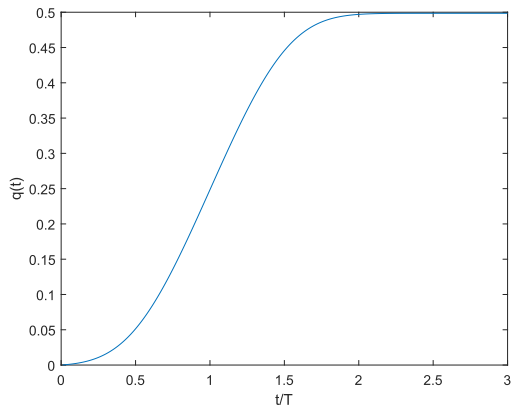

Fig. 3. The pulse-shaping function of the GFSK signal with $BT = 0.5$.

where

$$\phi_n = \pi \mathsf{h} \sum_{m=1}^{n-1} I_m + 2\pi \mathsf{h} I_n q(T). \tag{3}$$

Here $I_n \in \{0, 1\}$ is the information bits, h is the modulation index, and the pulse shaping function

$$q(t) = \frac{\int_0^t Q\left(2\pi B \frac{\tau - \frac{T}{2}}{\sqrt{\ln 2}}\right) - Q\left(2\pi B \frac{\tau + \frac{T}{2}}{\sqrt{\ln 2}}\right) d\tau}{2T} \tag{4}$$

with $Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty e^{\frac{-\tau^2}{2}} d\tau$.

If $\mathsf{h} = 0.5$ (as specified in [28]), (3) becomes

$$\phi_n = \theta_n + \pi I_n q(T), \tag{5}$$

where $\theta_n = \frac{\pi}{2} \sum_{m=1}^{n-1} I_m$. Here we set the frequency-time product $BT = 0.5$ (also as specified in [28]). As shown in Fig. 3, $q(t) = 0$ for $t \leq 0$, and $q(t) \approx 0.5$ for $t \geq 2T$.

We can model the continuous phase modulation as a process of finite state transition based on the phase transition (it is θ_n not ϕ_n) as shown in Fig. 4. Denote (r_i, r_j) as the state transition from $r_i \in \mathcal{S}$ at time nT to $r_j \in \mathcal{S}$ at time $(n+1)T$ driven by I_n. Here $\mathcal{S}$ stands for the set of the states of the trellis. The dash branches correspond to $I_n = -1$, the solid branches correspond to $I_n = 1$, and the associated signal is

$$x_n = e^{j(\theta_n + \pi I_n q(T))} = e^{j(\theta_n + \frac{\pi}{4} I_n)}, \tag{6}$$

where we have used the fact that $q(T) = \frac{1}{4}$.

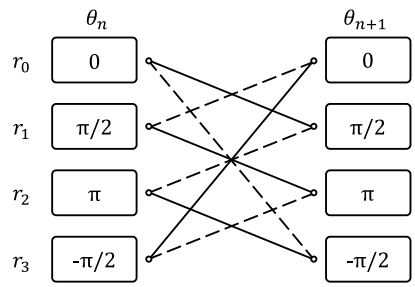

Fig. 4. The phase transition of the θ_n. The dash line means the input $I_n = -1 (b_n = 0)$, solid line means $I_n = 1 (b_n = 1)$.

If the input to the ISI channel (1) is an GFSK signal, we can combine the GFSK modulation and the multi-path channels into a (larger) trellis diagram.

C. Primer of the Viterbi and BCJR Algorithms

Based on the trellis diagram, the receiver can use the Viterbi algorithm or the BCJR algorithm for optimal symbol detection. We briefly review them to make this paper self-contained.

The Viterbi algorithm is for maximum likelihood (ML) detection. By exploiting the Markovian structure of the finite-memory channel, it computes the likelihood of each branch, i.e., the conditional probability density function (PDF) of the channel output given the inputs

$$p(\mathbf{y}_{1:N}|\mathbf{x}) = \prod_{k=1}^{N} p(y_k|\mathbf{x}_{k-L+1}^k), \tag{7}$$

where $x_k = 0, k < 0$. For a given channel output $\mathbf{y}$, we should have

$$\arg\max_{x_k \in \mathcal{X}} p(\mathbf{y}_{1:N}|\mathbf{x}) = \arg\min_{x_k \in \mathcal{X}} -\log p(\mathbf{y}_{1:N}|\mathbf{x}). \tag{8}$$

Combining (8) and (7), we can rewrite (7) as:

$$\arg\min_{x_k \in \mathcal{X}} -\log p(\mathbf{y}_{1:N}|x) = \arg\min_{x_k \in \mathcal{X}} -\sum_{k=1}^{N} \log p(y_k|s_k, s_{k+1}) \tag{9}$$

where $p(y_k|s_k, s_{k+1})$ is the likelihood of y_k associated with the state transition from s_k to s_{k+1}. The Viterbi algorithm can solve (9) efficiently by searching for the shortest path across the trellis diagram.

The BCJR algorithm [26] is for maximum *a posteriori* probability (MAP) detection. It computes the *a posteriori* log-likelihood ratio (LLR) of bits $\{u_k\}$

$$\begin{aligned} L(u_k|\mathbf{y}) &= \log \frac{p(u_k = 1|\mathbf{y})}{p(u_k = 0|\mathbf{y})} = \log \frac{p(u_k = 1, \mathbf{y})}{p(u_k = 0, \mathbf{y})} \\ &= \log \frac{\sum_{(s_k, s_{k+1}) \in \mathcal{S}^+} p(s_k, s_{k+1}, \mathbf{y})}{\sum_{(s_k, s_{k+1}) \in \mathcal{S}^-} p(s_k, s_{k+1}, \mathbf{y})}, \end{aligned} \tag{10}$$

where $\mathcal{S}^+$, $\mathcal{S}^-$ are the set of the ordered pairs (s_k, s_{k+1}) corresponding to all states transitions driven by $u_k = 1$ and $u_k = 0$, respectively. The sequence $\mathbf{y}$ in $p(s_k, s_{k+1}, \mathbf{y})$ can be written as $p(s_k, s_{k+1}, (y_1, \cdots, y_{k-1}), y_k, (y_{k+1}, \cdots, y_N))$. Applying the chain rule for joint probabilities, we can decompose $p(s_k, s_{k+1}, \mathbf{y})$ into:

$$p(s_k, s_{k+1}, \mathbf{y}) = \alpha_k(s_k) \gamma_k(s_k, s_{k+1}) \beta_{k+1}(s_{k+1}), \tag{11}$$

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

where $\alpha_k(s_k) \triangleq p(s_k, y_{1:k-1})$, $\gamma_k(s_k, s_{k+1}) \triangleq p(y_k, s_{k+1}|s_k)$, and $\beta_{k+1}(s_{k+1}) \triangleq p(y_{k+1:N}|s_{k+1})$. They can be recursively computed as

$$\alpha_k(s_k) = \sum_{s_{k-1}} \alpha_{k-1}(s_{k-1})\gamma_{k-1}(s_{k-1}, s_k), \quad (12a)$$

$$\beta_k(s_k) = \sum_{s_{k+1}} \beta_{k+1}(s_{k+1})\gamma_{k+1}(s_k, s_{k+1}), \quad (12b)$$

$$\gamma_k(s_k, s_{k+1}) = p(s_{k+1}|s_k) \cdot p(y_k|s_k, s_{k+1}), \quad (12c)$$

with the initialization $\alpha_0(s_0) = 1$ and $\beta_N(s_N) = 1$.

Driven by u_k, the probability of the state transition is

$$p(s_{k+1}|s_k) = \begin{cases} p(u_k = 1), & (s_k, s_{k+1}) \in \mathcal{S}^+, \\ p(u_k = 0), & (s_k, s_{k+1}) \in \mathcal{S}^-, \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

D. About the Likelihood Function

Examining (9) and (12), we see that the likelihood $p(y_k|s_k, s_{k+1})$ plays a key role in both Viterbi and BCJR algorithms. Indeed, the Viterbi algorithm depends solely on the likelihoods, while the BCJR also exploits the *a priori* information on the state transition probability $p(s_{k+1}|s_k)$, which is independent of the CSI [cf. (12c) and (13)]. Hence, $p(y_k|s_k, s_{k+1})$ is the only CSI-dependent component for both algorithms.

The conventional method calculates $p(y_k|s_k, s_{k+1})$ assuming known CSI and that the channel noise is zero-mean Gaussian with variance σ^2. Hence the likelihood can be computed to be

$$p(y_k|s_k, s_{k+1}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{\frac{-|y_k - v_k|^2}{2\sigma^2}\right\}, \quad (14)$$

where $v_k = \sum_{l=0}^{L-1} h_l x_{k-l}$ is the channel output associated with the state transition from s_k to s_{k+1} (cf. Fig. 2).

But when the CSI is unknown or the distribution of the noise is unknown owing to non-Gaussian co-channel interference, the model-based likelihood (14) will be erroneous, causing severe degradation to the performance of Viterbi or BCJR. To address this issue, we propose to use an ANN to learn online the likelihood function $p(y_k|s_k, s_{k+1})$ of the trellis diagram based on a pilot sequence, assuming no CSI nor the noise statistics.

III. The OLTD-Based Viterbi and BCJR

This section explains how the OLTD-based Viterbi and BCJR algorithms work. Both algorithms consist of three stages as explained in the following.

A. Stage One: Train ANN to Learn the Trellis Diagram

As shown in Fig. 5, we construct a fully connected and one hidden-layered ANN [35], whose input is the real and imaginary part of the received sample y_k and the output nodes correspond to all the state transitions in the trellis diagram. For example, we can use a network with 8 output nodes to learn the trellis diagram in Fig. 4, which has 8 state transitions. The hidden-layer nodes adopt the Sigmoid activation function,

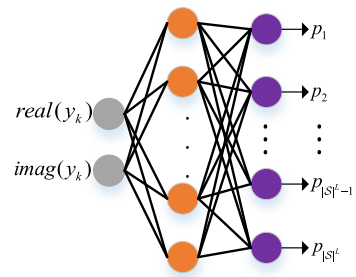

Fig. 5. The one-hidden layer ANN for learning the normalized likelihoods corresponding to the state transitions of the trellis diagram.

while the output layer employs the *Softmax* activate function. The reason of using the softmax function is to be explained in Section III-D.

Given the pilot sequence $\{y_k, k = 1, \ldots, P\}$, we know the true transition $s_k \to s_{k+1}$ corresponding to each y_k and thus label the normalized likelihood $p(y_k|s_k, s_{k+1})$ corresponding to the true transition to be 1 and all the others to be 0; this is the so-called *one-hot* encoding. The ANN is optimized according to the minimum cross-entropy criterion. Note that the OLTD method requires neither the CSI nor the noise statistics, which is its main advantage over the model-based method.

B. Stage Two: Feed the Payload Into the ANN

After the ANN is trained, the payload signals $\{y_i, i = P+1, \ldots, P+Q\}$ are then fed into the network; for each y_i the ANN will yield the normalized likelihoods $p(y_i|s_i, s_{i+1})$ for all the $2|\mathcal{S}|$ state transitions. The numerical examples in Section V indicates that the likelihoods yielded by the ANN is sufficiently accurate when it is trained with a pilot with length P no more than a few hundred.

C. Stage Three: Feed the Likelihoods Into the Viterbi or BCJR Algorithm

Given the likelihoods $p(y_i|s_i, s_{i+1})$'s per received sample of the payload, the Viterbi (or BCJR) algorithm can then be directly applied to obtain the ML (or MAP) detection.

Taking the BCJR algorithm for example, given the sample y_k's associated likelihoods $p(y_k|s_k, s_{k+1})$'s, the BCJR algorithm can calculate γ_k according to (12c) and further compute the forward recursion (12a) and the backward recursion (12b) to obtain the LLR $L(u_k|\mathbf{y})$ by (11) and (10). The combination of the OLTD method and the BCJR algorithm is termed as the OLTD-based BCJR. The OLTD-based Viterbi is even simpler: it just needs to search the most likelihood trellis path according to (9).

D. Discussions

Observe from (9) that to multiply $p(y_k|s_k, s_{k+1})$'s by a common positive factor does not affect the output result of the Viterbi algorithm, neither does it affect the BCJR as can be seen from (10) and (12). Hence, instead of learning the original likelihoods, which can be anywhere in $[0, \infty)$, the ANN can simply learn the normalized likelihoods

$$\frac{p(y_k|s_k, s_{k+1})}{\sum_{(s_k, s_{k+1}) \in \mathcal{S}^- \cup \mathcal{S}^+} p(y_k|s_k, s_{k+1})} \in [0, 1],$$

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

which is why we can adopt the softmax as the activation function of the output layer even though the actual likelihood value may be out of the range $[0, 1]$. The above insight is the underlying reason why our method is significantly simpler than the state-of-the-art method [24]. Indeed, the ViterbiNet algorithm first uses a deep neural network to learn the conditional probability $p(s_k, s_{k+1}|y_k)$, and then computes the likelihoods

$$p(y_k|s_k, s_{k+1}) = \frac{p(s_k, s_{k+1}|y_k)p(y_k)}{p(s_k, s_{k+1})}, \tag{15}$$

where the marginal distribution $p(y_k)$ was obtained in [24]) using a mixture model with a expectation minimization (EM) algorithm. Because of this complication, the ViterbiNet requires much more training data than the OLTD. Furthermore, $p(s_k, s_{k+1})$ in the denominator of (15) was assumed to be a constant (cf. [24, eq. (6)]), which is usually untrue, especially when the channel inputs are coded bits. We will revisit this point in Section IV-B.

It follows from (15) that the normalized likelihoods are exactly $p(s_k, s_{k+1}|y_k)$'s under the assumption of constant $p(s_k, s_{k+1})$. This observation suggests that the ViterbiNet could have forgone the computation of the marginal distribution $p(y_k)$ and achieve the same performance as the OLTD, if $p(s_k, s_{k+1})$ is indeed a constant for different state transitions.

We can gain more insights into the OLTD method by drawing its analogy to the classic least square (LS) fitting method. An LS method optimizes the parameters of a signal model to fit the true signal; the OLTD method trains the ANN to approximate the ground truth of the normalized likelihoods, which is a one-hot vector in absence of the channel noise. The LS method uses the LS criteria; the OLTD adopts the minimum cross-entropy criterion. The LS method does not assume knowledge of the noise statistics, neither does the OLTD, which may explain why the OLTD is robust against non-Gaussian noise as shown in Section IV.

We use an example to show how the one-hidden layer ANN as shown in Fig. 5 perform under different pilot length. Consider an ISI channel with impulse response $h(n) = 0.407\delta(n) + 0.815\delta(n-1) + 0.407\delta(n-2)$ with BPSK inputs, whose trellis diagram is as shown in Fig. 2. Given complex-valued Gaussian noise with variance σ^2, the normalized likelihoods are proportional to $p_i(y_k) \propto \frac{1}{\pi\sigma}\exp\{-|y_k - a_i|^2/\sigma^2\}$, where $a_i, i = 1, \ldots, 8$ are as labeled on the state-transition branches in Fig. 2. After being trained by a pilot sequence using the one-hot vector as explained in Section III-A, the ANN is essentially a learned mapping $q_i(y_k), i = 1, \ldots, 8$. We train the ANN using pilots with different lengths under various SNRs, i.e., $5, 10$, and 12dB. Fig. 6 shows the expected Kullback-Leibler (KL) divergence $D(p||q) = \mathbb{E}_y[\sum_{i=1}^{8} p_i(y)\log p_i(y)/q_i(y)]$, where y as the output of the ISI channel is of Gaussian mixture distribution. The KL divergence diminishes as the pilot length increases to a few hundred. It is not surprising that the ANN can learn the trellis diagram well, since a ANN with even one hidden layer can be a universal approximator [36].

Last but not least, our proposed OLTD-based BCJR algorithm can be readily applied for turbo equalization [37], which is an important advantage over the BCJRNet [23] as we will explain in the next.

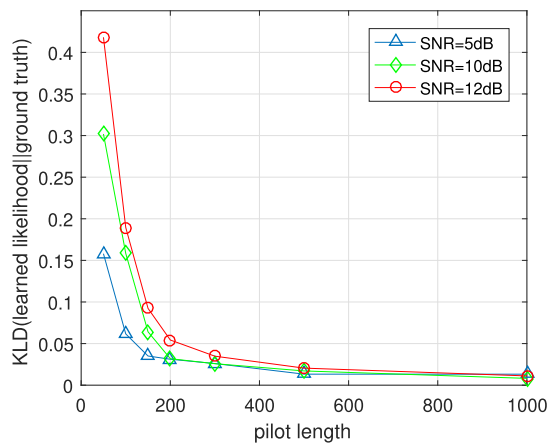

Fig. 6. The Kullback-Leibler (KL) divergence between p and q under different SNRs and different pilot lengths.

Fig. 7. A bit-interleaved coded GFSK transmitter, which may be a potential enhancement to the PHY of BLE 5.0.

IV. The OLTD-Based Turbo Equalization for a Coded GFSK System

In this section, we apply the OLTD method for turbo equalization, for which a bit interleaver is inserted between the forward error correction (FEC) encoder and the modulator, as shown in Fig. 7. This section serves for two purposes: i) the OLTD method is shown to be able to be incorporated seamlessly into a turbo equalizer for superb performance; ii) we advocate a potential enhancement to the current BLE standard for significantly better receiver sensitivity, as an interesting byproduct of this study.

Consider that the FEC encoder in Fig. 7 is the convolutional code whose generator polynomials are

$$\begin{aligned} G_0(x) &= 1 + x + x^2 + x^3, \\ G_1(x) &= 1 + x^2 + x^3, \end{aligned} \tag{16}$$

which is actually the convolutional code adopted in the BLE standard [30], and the modulation is the GFSK (as explained in Section II-B). The information bits are denoted as $\{u_k\}$, the coded bits $\{a_n\}$, and the interleaved bits $\{b_n\}$, which will be modulated into symbols $\{x_n\}$.

A. OLTD-Based Turbo Equalization

The whole procedure of turbo equalization is as shown in Fig. 8, where BCJR equalization and BCJR decoding are conducted iteratively.[1]

On the equalization side, the BCJR algorithm takes the a priori probabilities of b_n's to calculate the state transition

[1]Readers unfamiliar with turbo equalization may refer to [37] for an excellent tutorial.

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

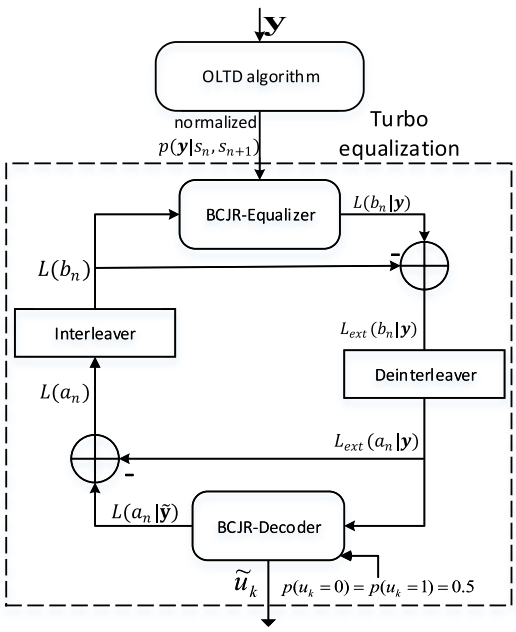

Fig. 8. Overview of turbo equalization. The OLTD-based turbo equalization differs from the the model-based one only in that the normalized likelihood is obtained using the OLTD algorithm.

probabilities as

$$p(s_{n+1}|s_n) = \begin{cases} p(b_n = 0), & \text{if } b_n = 0 \text{ drives } s_n \to s_{n+1} \\ p(b_n = 1), & \text{if } b_n = 1 \text{ drives } s_n \to s_{n+1}, \end{cases} \tag{17}$$

takes the normalized likelihoods from the OLTD algorithm, and then calculate (12) and (11) to calculate $L(b_n|\mathbf{y})$ in (10).

Note that the a posteriori LLR of $\{b_n\}$ can also be computed as

$$\begin{aligned} L(b_n|\mathbf{y}) &= \log \frac{\sum_{\forall \mathbf{b}: b_n=1} p(\mathbf{b}, \mathbf{y})}{\sum_{\forall \mathbf{b}: b_n=0} p(\mathbf{b}, \mathbf{y})} \\ &= \log \frac{\sum_{\forall \mathbf{b}: b_n=1} p(\mathbf{y}|\mathbf{b}) p(\mathbf{b})}{\sum_{\forall \mathbf{b}: b_n=0} p(\mathbf{y}|\mathbf{b}) p(\mathbf{b})}. \end{aligned} \tag{18}$$

Since the bit interleaver decorrelates the neighboring coded bits, it holds that $p(\mathbf{b}) = \prod_{n=1}^{N} p(b_n)$; thus, $L(b_n|\mathbf{y})$ can be decomposed into

$$L(b_n|\mathbf{y}) = L_{ext}(b_n|\mathbf{y}) + L(b_n), \tag{19}$$

where

$$L_{ext}(b_n|\mathbf{y}) = \log \frac{\sum_{\forall \mathbf{b}: b_n=1} p(\mathbf{y}|\mathbf{b}) \prod_{i=1: i \neq n}^{N} p(b_i)}{\sum_{\forall \mathbf{b}: b_n=0} p(\mathbf{y}|\mathbf{b}) \prod_{i=1: i \neq n}^{N} p(b_i)} \tag{20}$$

is the extrinsic information about b_n contained in $\mathbf{y}$, and

$$L(b_n) = \log \frac{p(b_n = 1)}{p(b_n = 0)} \tag{21}$$

is called the intrinsic information. Hence, it follows from (19) that

$$L_{ext}(b_n|\mathbf{y}) = L(b_n|\mathbf{y}) - L(b_n), \quad n = 1, 2, \ldots, N. \tag{22}$$

On the decoding side, after deinterleaving $L_{ext}(b_n|\mathbf{y})$ into $L_{ext}(a_n|\mathbf{y})$, the BCJR decoder takes the extrinsic information as the "received signal", i.e., $\tilde{\mathbf{y}} \triangleq L_{ext}(a_n|\mathbf{y})$ to update the a posteriori LLR of the coded bits $L(a_n|\tilde{\mathbf{y}})$ based on the trellis diagram associated with the FEC (16). As another input into the BCJR decoder, the uncoded bits $\{u_k, k = 1, \ldots, K\}$ are assumed to satisfy $P(u_k = 1) = P(u_k = 0) = 1/2$.

Note from (19) that we can update the intrinsic LLRs

$$L(a_n) = L(a_n|\tilde{\mathbf{y}}) - L_{ext}(a_n|\mathbf{y}), \tag{23}$$

which can be fed into the BCJR equalizer after being interleaved into $L(b_n)$. Using the relationship [cf. (21)]

$$p(b_n) = \begin{cases} \dfrac{1}{1 + e^{L(b_n)}}, & \text{if } b_n = 0, \\ \dfrac{e^{L(b_n)}}{1 + e^{L(b_n)}}, & \text{if } b_n = 1 \end{cases} \tag{24}$$

and (17), we can obtain the state-transition probabilities $p(s_{n+1}|s_n)$'s, which are needed for the next round of BCJR equalization.

After a prescribed number of iterations, we calculate $\tilde{\gamma}_k(r_i, r_j)$ as [37, eq.(22)] (25), shown at the bottom of the next page, for $k = 1, 2, \cdots, K$, and the decoder will calculate the LLRs of information bits u_k, and make a final decision

$$\tilde{u}_k = \begin{cases} 0, & \text{if } L(u_k|\tilde{\mathbf{y}}) < 0, \\ 1, & \text{if } L(u_k|\tilde{\mathbf{y}}) \geq 0. \end{cases} \tag{26}$$

Hence, the only difference between the OLTD-based turbo equalization and a standard model-based one is how the likelihoods $p(\mathbf{y}|s_n, s_{n+1})$ are obtained, while the whole procedure in the dotted-line box is the same for both methods.

B. Apply BCJRNet to Turbo Equalization?

One may attempt to apply the BCJRNet to turbo equalization, but will come across a major issue as explained in the next.

In the BCJRNet, a neural network is trained to obtain the a posteriori probability $p(s_n, s_{n+1}|y_n)$. Then by Bayes' rule

$$p(y_n|s_n, s_{n+1}) = \frac{p(s_n, s_{n+1}|y_n) p(y_n)}{p(s_{n+1}, s_n)}. \tag{27}$$

Combining (27) and (12c), we obtain

$$\gamma_n(s_n, s_{n+1}) = \frac{p(s_n, s_{n+1}|y_n) p(y_n)}{p(s_n)}. \tag{28}$$

According to (10) and (11), we have

$$\begin{aligned} &L(b_n|\mathbf{y}) \\ &= \log \frac{\sum_{(s_n, s_{n+1}) \in \mathcal{S}^+} \alpha_n(s_n) \gamma_n(s_n, s_{n+1}) \beta_{n+1}(s_{n+1})}{\sum_{(s_n, s_{n+1}) \in \mathcal{S}^-} \alpha_n(s_n) \gamma_n(s_n, s_{n+1}) \beta_{n+1}(s_{n+1})}. \end{aligned} \tag{29}$$

Substituting (28) into (29) yields

$$\begin{aligned} &L(b_n|\mathbf{y}) \\ &= \log \frac{\sum_{(s_n, s_{n+1}) \in \mathcal{S}^+} \alpha_n \cdot \frac{p(s_n, s_{n+1}|y_n) p(y_n)}{p(s_n)} \cdot \beta_{n+1}}{\sum_{(s_n, s_{n+1}) \in \mathcal{S}^-} \alpha_n \cdot \frac{p(s_n, s_{n+1}|y_n) p(y_n)}{p(s_n)} \cdot \beta_{n+1}}, \\ &= \log \frac{\sum_{(s_n, s_{n+1}) \in \mathcal{S}^+} \alpha_n \cdot \frac{p(s_n, s_{n+1}|y_n)}{p(s_n)} \cdot \beta_{n+1}}{\sum_{(s_n, s_{n+1}) \in \mathcal{S}^-} \alpha_n \cdot \frac{p(s_n, s_{n+1}|y_n)}{p(s_n)} \cdot \beta_{n+1}}, \end{aligned} \tag{30}$$

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

Given the a posteriori probability $p(s_n, s_{n+1}|y_n)$ learned by the BCJRNet, one can update $p(s_n)$ through the forward recursion

$$p(s_n) = \sum_{s_{n-1}\in\mathcal{S}} p(s_n|s_{n-1})p(s_{n-1}), \tag{31}$$

where $p(s_n|s_{n-1})$ can be obtained according to (17) and (24). Due to this recursion, the BCJR-based turbo equalization is cumbersome.

More important, this method actually does not work as shown by the simulation example in Section V-B. Indeed, the a posteriori probability $p(s_n, s_{n+1}|y_n)$ is not the suitable metric to learn, because

$$p(s_n, s_{n+1}|y_n) \propto p(y_n|s_n, s_{n+1})p(s_n, s_{n+1}) \tag{32}$$

relies on the a priori information $p(s_n, s_{n+1})$; thus, it is impossible to infer $p(s_n, s_{n+1}|y_n)$ from y_n itself unless $p(s_n, s_{n+1})$ is a constant, which is usually untrue. In contrast, the likelihood $p(y_n|s_n, s_{n+1})$ relies solely on the distribution of the channel noise and the CSI, and is independent of the channel coding; thus, it can be learned from y_n itself. Indeed, to obtain $p(y_n|s_n, s_{n+1})$, the BCJRNet algorithm assumes that the inputs into the channel are equiprobable so that $p(s_{n+1}, s_n) = |\mathcal{S}|^{-L}$ is a constant [23, eq. (13)], which conflicts with the basic idea of a Turbo equalization. That is, to utilize the a priori probabilities of b_n's from the FEC decoder for improved equalization output.

V. Simulation Results

In this section, we present simulation examples to validate the feasibility and superior performance of the OLTD method applied to the Viterbi algorithm, the BCJR algorithm, and the turbo equalization.

We adopt a fully-connected neural network with a single hidden layer as shown in Fig. 5 for the OLTD. The hidden layer has 100 neurons and employs the *Sigmoid* activate function. The number of neurons in the output layer is the same as the number of state transitions in the trellis diagram. The likelihood of each state transition is normalized by the *Softmax* to approximate the ground truth, i.e., the one hot vector. The network is trained using the Adamax optimizer [38] to minimize the cross-entropy based on a pilot sequence. The pilot length is set to be 500 in the simulations about the QPSK signals and is set to be 256 in the GFSK simulations. The optimizer divides the pilot into mini-batches of 16 samples and the initial learning rate is set to be 0.01. The ANN is trained 200 times based on the same pilot. That is, the total amount of training pilot is only 500 samples (or 256 samples in the GFSK examples). The settings of our training conditions are summarized in Table I.

TABLE I
The Simulation Setting

NN Toolkit	Keras using Tensorflow backend
Training Processor	Inter(R) i7-6700 CPU
Training Batch Size	16
Training Epoch	200
No. of Hidden Layer	1 (100 neurons)
Optimizer	Adamax
Activate Function	Sigmoid—Softmax
Loss Function	Cross Entropy
Pilot(Training) Length	500 in QPSK and OOK, 256 in GFSK cases

Three types of signals are simulated: i) uncoded Quadrature Phase Shift Keying (QPSK) transmitted over an ISI channel, ii) bit-interleaved On-Off Keying (OOK) in a Poisson channel, and iii) coded GFSK as adopted in the PHY of the BLE – all can be represented by a trellis diagram.[2]

A. Uncoded QPSK in an ISI Channel With Additive Noise

We first simulate an ISI channel as modelled in (1), where the input signal is uncoded QPSK, the noise is complex-valued Gaussian, and channel coefficients are given by $h_l \triangleq \sqrt{\frac{e^{-\gamma\cdot l}}{\sum_{l=0}^{L-1} e^{-\gamma\cdot l}}}$ for $\gamma > 0$. We set the channel memory length $L = 2$; thus, the trellis diagram is fully-connected with 4 states and 16 branches (state transitions). Fig. 9 compares the bit error rate (BER) performance of model-based BCJR/Viterbi algorithm and OLTD-based BCJR/Viterbi algorithm. The model-based method is simulated based on perfect CSI, while the OLTD is trained based on a 500-sample pilot. The simulation results are obtained by averaging over $10,000$ Monte-Carlo simulations, where the channel coefficients are generated with γ being draw at random in the range $[0.1, 1]$. Fig. 9 shows that OLTD-based method can achieve performance very close to the model-based benchmark. The Viterbi algorithm performs the same as the BCJR algorithm, since here the bits are generated 0 or 1 evenly.

We then simulate the channel noise as complex Cauchy with the PDF

$$f(z_k) = \frac{1}{\pi^2} \cdot \frac{\lambda^2}{(\mathrm{Re}\{z_k\}^2 + \lambda^2) \cdot (\mathrm{Im}\{z_k\}^2 + \lambda^2)}, \tag{33}$$

where $\mathrm{Re}\{\cdot\}$ and $\mathrm{Im}\{\cdot\}$ stand for the real and imaginary parts, respectively. The Cauchy's PDF has much heavier tails than the Gaussian's. In the Cauchy noise case, the SNR is undefined since the variance of Cauchy is unbounded. We simulate the BER performance by changing the value of γ. Fig. 10 shows that the OLTD-BCJR outperforms the model-based BCJR algorithm that erroneously assumes Gaussian noise, which is not surprising since the OLTD requires no *a priori* knowledge on the statistics of the noise. The x-axis of Fig. 10 is different

[2]The Matlab^{TM} codes used for generating the simulation results can be found: https://github.com/csrlab-fudan/OLTD-code.

$$\tilde{\gamma}_k(r_i, r_j) = \begin{cases} p(u_k) \cdot p(a_{1,i,j}|\mathbf{y}) \cdot p(a_{2,i,j}|\mathbf{y}), & \text{if } (s_n, s_{n+1}) \in \mathcal{S}^+ \cup \mathcal{S}^-, \\ 0, & \text{if } (s_n, s_{n+1}) \notin \mathcal{S}^+ \cup \mathcal{S}^-, \end{cases} \tag{25}$$

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

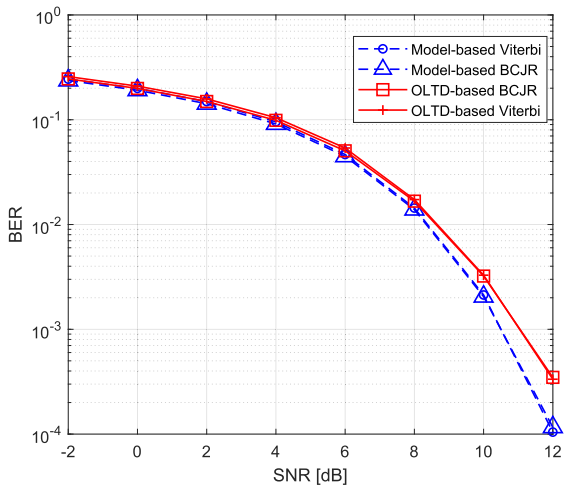

Fig. 9. The BER performance of the model-based and OLTD-based receiver for uncoded QPSK.

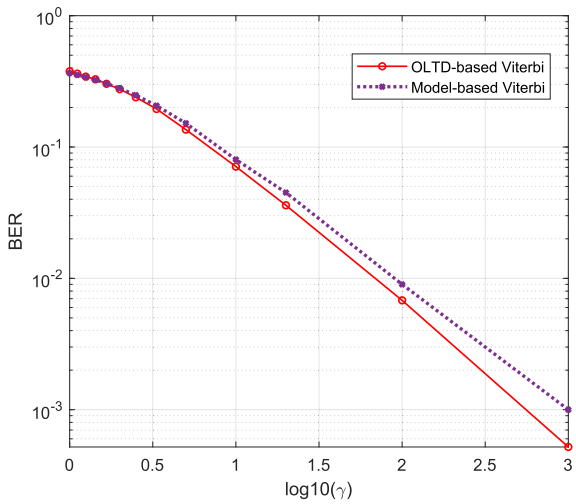

Fig. 10. The BER performance of the OLTD-Viterbi versus the model-based Viterbi receiver, which assumes Gaussian noise while the noise is of Cauchy distribution.

because the variance of Cauchy noise is unbounded; thus, the standard definition of SNR does not apply here.

We also simulate the ISI channel scenario where the QPSK signal is interfered by a random 4-ary Pulse Amplitude Modulation (4-PAM) source. The received power of the interference is the same as the QPSK signal, i.e., the signal-to-interference ratio (SIR) is 0 dB. As shown in Fig. 11, the model-based Viterbi method based on the assumption of Gaussian noise fails. The striking advantage of the OLTD-based approach indicates that the neural network somehow learned the "structure" of the non-Gaussian interference and hence suppressed it effectively.

To demonstrate the influence of training pilot length on reception performance, we simulate the BER performance of the OLTD-based approach when trained using the pilots length of $\{0.5, 1, 2, 5, 10, 20\} \times 10^2$ as shown in Fig. 12. We set SNR $=$ 10 dB under Gaussian channel, and the performance of the OLTD-based method, as the training pilots length increases, gradually approaches that of the model-based method under perfect CSI. A pilot of length a few hundred

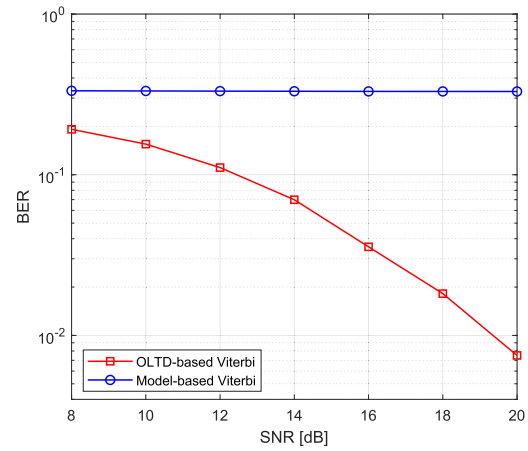

Fig. 11. The BER performance of the OLTD-based Viterbi compared with the model-based method. The signal is subject to co-channel 4-PAM interference, while the model-based method mistakenly regards the interference-plus-noise as Gaussian.

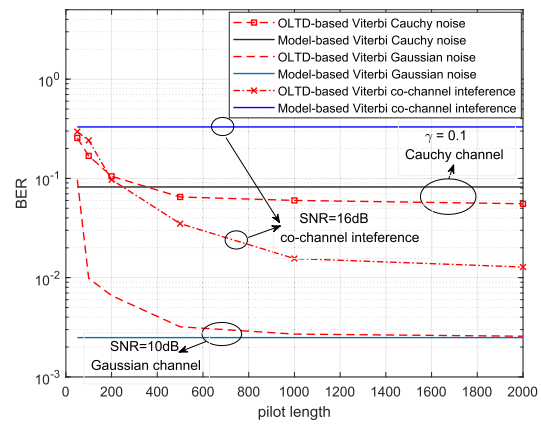

Fig. 12. The BER performance versus the length of pilots in the SNR value 10dB.

is sufficient for all the three cases. Fig. 12 illustrates from a different perspective that the OLTD-based method can outperform the model-based method in the two cases of non-Gaussian noise.

B. Bit-Interleaved Coded OOK in a Poisson Channel

In addition to the additive noise channel, we also simulate the Poisson channel as previously considered in [23]. But here we consider a bit-interleaved system as shown in Fig. 7, where the information bits are FEC encoded, bit-interleaved, and then modulated by OOK before being transmitted over a Poisson channel. The channel output y_k is of Poisson distribution, i.e.,

$$p(y_k|s_k, s_{k+1}) = \frac{(v+1)^{y_k}}{y_k!} e^{-(v+1)}, \quad y_k = 0, 1, 2, \ldots \tag{34}$$

where $v = \sum_{l=0}^{L-1} h_l x_{k-l}$ with $x_{k-l} \in \{0, 1\}$ and $h_l \triangleq \sqrt{\frac{e^{-\gamma \cdot l}}{\sum_{l=0}^{L-1} e^{-\gamma \cdot l}}}$ for γ being draw at random in the range$[0.1, 1]$.

For $L = 2$, the Poisson channel can be represented by a fully-connected two-state trellis diagram with 4 branches. Hence, the OLTD uses a neural network with 4 outputs. Fig. 13 shows that using no iterations the OLTD-based method and the

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

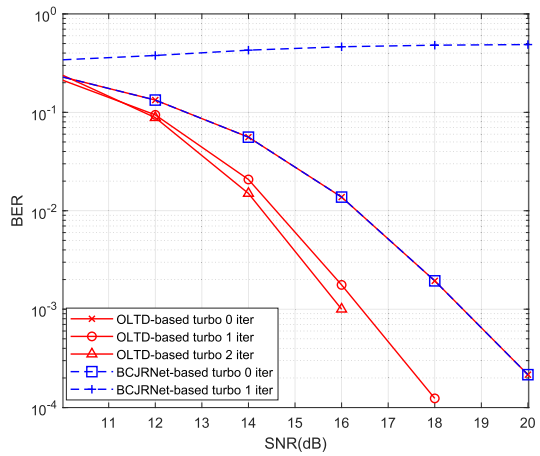

Fig. 13. The BER performance of the OLTD-based turbo equalization compared to the BCJRNet-based turbo in Poisson channel.

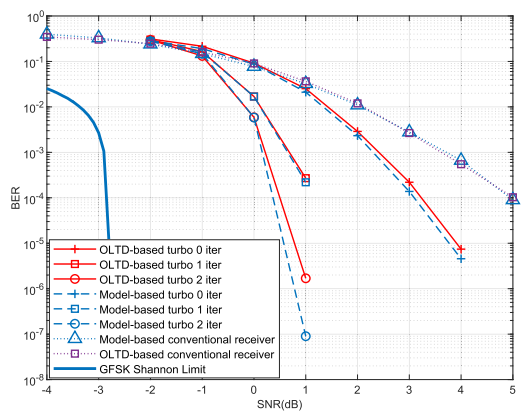

Fig. 14. The performances of the model-based and OLTD-based BLE receiver in an AWGN channel versus the performance of the enhanced BLE system with bit-interleaving.

BCJRNet have identical performance. The BCJRNet applied for turbo equalization, however, leads to failed decoding as explained in Section IV-B. In contrast, the OLTD-based turbo equalization can achieve significantly improved performance.

C. A BLE System and Its Enhancement

We simulate in an AWGN channel a BLE system with bit rate 500 Kbps with a pilot of length 256 as specified in the protocol [30]. A model-based receiver consists of a BCJR demodulator and a Viterbi decoder. It assumes perfect CSI. The OLTD-based receiver differs from the model-based one only in that the likelihoods $p(y_k|s_k, s_{k+1})$ [cf. (12c)] used in the BCJR algorithm is produced by a neural network trained by the 256-sample pilot sequence. According to Fig. 4, the GFSK modulation can be represented by a trellis diagram with 8 state transitions; thus, the neural network for the OLTD has 8 neurons in the output layer. The BER is averaged over $10,000$ Monte Carlo trails of the channel coefficients $h(\theta) = e^{j\theta}$, with θ being drawn at random in the range $[0, 2\pi]$. In Fig. 14, the dot dash line with marker $\triangle$ corresponds to the model-based method using perfect CSI, while the dot dash line with marker $\square$ is the OLTD-based one. They essentially overlap, which suggests that the neural network

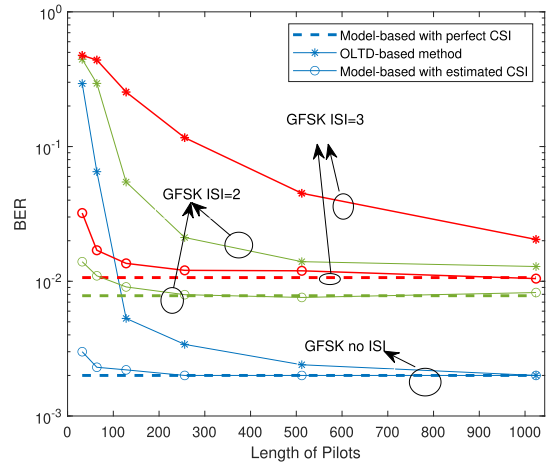

Fig. 15. The impact of the pilot length on the BER performance of the OLTD-based receiver in the ISI channels with different memory.

assisted receiver can be practically feasible at least performance-wise.

We also present Shannon limit of the BER performance of the GFSK signal obtained using the numerical method in [39] and [40]. The large gap between Shannon limit and the achieved performance of the BLE system motivated us to consider introducing a bit interleaver at the transmitter between encoder and GFSK modulator (cf. Fig. 7). Given the bit-interleaver, the receiver can apply the turbo equalization. Fig. 14 also illustrates the BER performance of the turbo receiver with no iteration, and with 1 and 2 iterations. The three dash lines show the model-based turbo equalization under the perfect CSI and the other three sold lines with markers correspond to the OLTD-based turbo algorithm with pilot length of 256. It can be seen that introducing the bit-interleaver and using the turbo equalization in the receiver can yield $4 \sim 5$dB gain compared with the conventional receiver. Hence, to introduce bit-interleaving may be an interesting enhancement to the existing BLE protocol for significantly enhanced performance, which can make it more competitive for IoT communications.

In summary, it is seen from Fig. 9 and 14 that the BER performance of the data-driven OLTD method can only approach but not surpass the model-based method when the channel noise is Guassian. The OLTD sees prominent relative gain over the model-based method when the assumption made by the latter is erroneous, e.g., when the noise is non-Gaussian, as shown in Fig. 10, Fig. 11, and Fig. 12. Hence the OLTD is a robuster design and is preferable especially in the presence of non-Gaussian interferences.

In the last example, we simulate the BLE system in an L-length ISI channel generated as $h_0\delta(n) + \ldots + h_{L-1}\delta(n - L + 1)$, where the channel coefficients are i.i.d. complex-valued Gaussian but with the total power being normalized to 1. The combination of the GFSK modulation and the ISI channel can be modeled by a trellis diagram with 2^{L+1} states and 2^{L+2} state-transitions. For the ISI channel, the OLTD method can still work except that here the output layer of the neural network has 2^{L+2} neurons. Fig. 15 compares the BER performance of the OLTD-based algorithm $(- * -)$ with the

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

model-based receiver using estimated CSI ($-\circ-$) and perfect CSI ($--$) at SNR 2dB. It is seen that the OLTD requires a much longer pilot to approach to the genie-aided baseline, because the number of states to learn increases exponential with L. Hence, it calls for further researches on remedies for this issue.

VI. Conclusion

This paper introduced a method named online learning trellis diagram (OLTD), which uses a single hidden-layer artificial neural network (ANN) to learn the likelihoods of the received samples under different state transitions. It can be applied to replace only the channel state-dependent part of the Viterbi algorithm and the BCJR algorithm. We applied the OLTD-based Viterbi/BCJR algorithms to a coded QPSK/GFSK system, and the simulation results show that using a pilot sequence of length only a few hundred samples the OLTD based methods can perform similarly to their model-based counterpart given perfect channel state information (CSI) and Gaussian noise. In contrast to the model-based approaches, the OLTD-based approach assumes neither CSI nor statistics of the noise, which makes it robust against non-Gaussian interferences. In contrast to the state-of-the-art machine learning assisted methods, such as the BCJRNet and ViterbiNet, the proposed method does not assume the a priori probabilities of the coded bits, which makes it readily applicable to turbo equalization. The OLTD-based algorithms can be applied to the standard Bluetooth system and the enhanced one with bit-interleaving, because they require only some pilot of moderate length. Introducing bit-interleaving can be a beneficial enhancement to the BLE standard as an interesting by-product of this study. The proposed OLTD method also has a major drawback: learning of the trellis diagram requires a much longer pilot as the number of states increases exponential with the channel memory length. How to find a remedy for this issue would be an interesting research topic in the future.

References

[1] J. G. Proakis and M. Salehi, *Digital Communications*. New York, NY, USA: McGraw-Hill, 2008.

[2] T. J. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cogn. Commun. Netw.*, vol. 3, no. 4, pp. 563–575, Oct. 2017.

[3] Q. Mao, F. Hu, and Q. Hao, "Deep learning for intelligent wireless networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2595–2621, 4th Quart., 2018.

[4] D. Gunduz, P. de Kerret, N. Sidiroupoulos, D. Gesbert, C. Murthy, and M. van der Schaar, "Machine learning in the air," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2184–2199, Oct. 2019.

[5] N. H. Tran, W. Bao, A. Zomaya, M. N. H. Nguyen, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2019, pp. 1387–1395.

[6] S. Park, O. Simeone, and J. Kang, "Meta-learning to communicate: Fast end-to-end training for fading channels," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2020, pp. 5075–5079.

[7] E. Nachmani, Y. Be'ery, and D. Burshtein, "Learning to decode linear codes using deep learning," in *Proc. 54th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Sep. 2016, pp. 341–346.

[8] A. Bennatan, Y. Choukroun, and P. Kisilev, "Deep learning for decoding of linear codes—A syndrome-based approach," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2018, pp. 1595–1599.

[9] F. Liang, C. Shen, and F. Wu, "An iterative BP-CNN architecture for channel decoding," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 144–159, Feb. 2018.

[10] E. Nachmani, E. Marciano, L. Lugosch, W. J. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 119–131, Feb. 2018.

[11] T. Gruber, S. Cammerer, J. Hoydis, and S. T. Brink, "On deep learning-based channel decoding," in *Proc. 51st Annu. Conf. Inf. Sci. Syst. (CISS)*, Mar. 2017, pp. 1–6.

[12] N. Samuel, T. Diskin, and A. Wiesel, "Deep MIMO detection," in *Proc. IEEE 18th Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, Jul. 2017, pp. 1–5.

[13] N. Samuel, T. Diskin, and A. Wiesel, "Learning to detect," *IEEE Trans. Signal Process.*, vol. 67, no. 10, pp. 2554–2564, May 2019.

[14] J. Li et al., "Deep learning-based massive MIMO CSI feedback," in *Proc. 18th Int. Conf. Opt. Commun. Netw. (ICOCN)*, 2019, pp. 1–3.

[15] T. J. O'Shea, T. Erpek, and T. C. Clancy, "Deep learning based MIMO communications," 2017, *arXiv:1707.07980*.

[16] Y. Liao, N. Farsad, N. Shlezinger, Y. C. Eldar, and A. J. Goldsmith, "Deep neural network symbol detection for millimeter wave communications," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.

[17] N. Farsad and A. Goldsmith, "Neural network detection of data sequences in communication systems," *IEEE Trans. Signal Process.*, vol. 66, no. 21, pp. 5663–5678, Nov. 2018.

[18] F. A. Aoudia and J. Hoydis, "End-to-end learning of communications systems without a channel model," in *Proc. 52nd Asilomar Conf. Signals, Syst., Comput.*, Oct. 2018, pp. 298–303.

[19] H. Ye, L. Liang, G. Y. Li, and B.-H. F. Juang, "Deep learning-based end-to-end wireless communication systems with conditional GANs as unknown channels," *IEEE Trans. Wireless Commun.*, vol. 19, no. 5, pp. 3133–3143, May 2020.

[20] H. He, S. Jin, C. Wen, F. Gao, G. Y. Li, and Z. Xu, "Model-driven deep learning for physical layer communications," *IEEE Wireless Commun.*, vol. 26, no. 5, pp. 77–83, Oct. 2019.

[21] X. Gao, S. Jin, C.-K. Wen, and G. Y. Li, "ComNet: Combination of deep learning and expert knowledge in OFDM receivers," *IEEE Commun. Lett.*, vol. 22, no. 12, pp. 2627–2630, Dec. 2018.

[22] J. Liao, J. Zhao, F. Gao, and G. Y. Li, "A model-driven deep learning method for massive MIMO detection," *IEEE Commun. Lett.*, vol. 24, no. 8, pp. 1724–1728, Aug. 2020.

[23] N. Shlezinger, N. Farsad, Y. C. Eldar, and A. J. Goldsmith, "Data-driven factor graphs for deep symbol detection," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2020, pp. 2682–2687.

[24] N. Shlezinger, N. Farsad, Y. C. Eldar, and A. J. Goldsmith, "ViterbiNet: A deep learning based Viterbi algorithm for symbol detection," *IEEE Trans. Wireless Commun.*, vol. 19, no. 5, pp. 3319–3331, May 2020.

[25] G. D. J. Forney, "The Viterbi algorithm," *Proc. IEEE*, vol. 61, no. 5, pp. 268–278, Mar. 1993.

[26] L. Bahl, J. Cocke, F. Jelinek, and J. Raviv, "Optimal decoding of linear codes for minimizing symbol error rate," *IEEE Trans. Inf. Theory*, vol. IT-20, no. 2, pp. 284–287, Mar. 1974.

[27] G. Mclachlan and D. Peel, *Finite Mixture Model*, vol. 44. New York, NY, USA: Wiley, Jan. 2000.

[28] E. Au, "Bluetooth 5.0 and beyond [standards]," *IEEE Veh. Technol. Mag.*, vol. 14, no. 2, pp. 119–120, Jun. 2019.

[29] C.-E. Sundberg, "Continuous phase modulation," *IEEE Commun. Mag.*, vol. CM-24, no. 4, pp. 25–38, Apr. 1986.

[30] *Bluetooth Core Specification, version 5.0*. Accessed: Nov. 6, 2022. [Online]. Available: https://www.bluetooth.com/zh-cn/specifications/specs/core-specification-5/

[31] T. Okada and Y. Iwanami, "Turbo equalization of GMSK signals using noncoherent frequency detection," *IEICE Trans. Electron.*, vol. 85, no. 3, pp. 473–479, 2002.

[32] X. Wang and Z. Yang, "Turbo equalization for GMSK signaling over multipath channels," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.* , May 2001, pp. 2641–2644.

[33] T. Okada and Y. Iwanami, "Turbo equalization of GMSK signals using limiter-discriminator," in *Proc. ISSSE*, 2001, pp. 424–427.

[34] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Comput. Netw.*, vol. 54, no. 15, pp. 2787–2805, Jun. 2010.

[35] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, Sep. 2015.

[36] M. Stinchcombe and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, Jun. 1988.

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.

[37] R. Koetter, A. Singer, and M. Tuchler, "Turbo equalization," *IEEE Signal Process. Mag.*, vol. 21, no. 1, pp. 67–80, Jan. 2004.
[38] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015, pp. 1–15.
[39] L. I. Bing, F. Wei, B. M. Bai, and M. A. Xiao, "Fundamental performance limits of CPM coded modulation system," *J. Commun.*, vol. 35, no. 3, pp. 183–192, 2014.
[40] D. M. Arnold, H.-A. Loeliger, P. O. Vontobel, A. Kavcic, and W. Zeng, "Simulation-based computation of information rates for channels with memory," *IEEE Trans. Inf. Theory*, vol. 52, no. 8, pp. 3498–3508, Aug. 2006.

Jie Yang (Student Member, IEEE) received the M.S. degree in communication engineering from Fudan University, Shanghai, China, in 2022. He is currently a System Engineer of wireless communication with Huawei Company. His research interests include MIMO wireless communication and the application of neural network in wireless communication.

Qinghe Du (Student Member, IEEE) received the M.S. degree in communication engineering from Fudan University, Shanghai, China, in 2020, where she is currently pursuing the D.Eng. degree with the Department of Communication Science and Engineering. Her research interests include signal processing for wireless communication and physical layer design of wireless communication receiver.

Yi Jiang (Member, IEEE) received the B.S. degree in electrical engineering and information science from the University of Science and Technology of China (USTC), Hefei, China, in 2001, and the M.S. and Ph.D. degrees in electrical engineering from the University of Florida, Gainesville, FL, USA, in 2003 and 2005, respectively. In 2005, he was a Research Consultant at Information Science Technologies Inc., Fort Collins, CO, USA. From 2005 to 2007, he was a Post-Doctoral Researcher with the University of Colorado, Boulder, CO, USA. He moved to San Diego, CA, USA, in 2007, and worked for multiple companies, including NextWave Wireless from May 2007 to July 2008, Qualcomm Corporate Research and Development from August 2008 to May 2012, IAA Incorporated from June 2012 to January 2013, and Silvus Technologies from February 2013 to July 2016. He also worked as an Adjunct Researcher at the Electrical Engineering Department, University of California at Los Angeles, Los Angeles, CA, USA, from October 2014 to July 2016. He joined Fudan University in August 2016, where he is currently a Professor with the Department of Communication Science and Engineering, Fudan University. His research interests include array signal processing and mobile ad hoc networks.

Authorized licensed use limited to: University of Southern California. Downloaded on April 19,2026 at 23:15:47 UTC from IEEE Xplore. Restrictions apply.